

Compiler Design Lab Report 3

YACC-Based Arithmetic Expression Syntax Analyzer

Abdullah Al Jubaer Gem

ID: 210041226

Section: 2B

May 20, 2026

1 Objective

Design and implement a YACC-based syntax analyzer for arithmetic expressions. The parser recognizes and evaluates expressions supporting addition, subtraction, multiplication, division, exponentiation, and parentheses, while detecting syntax errors.

2 Expression Grammar

The grammar encodes operator precedence structurally — lower-precedence operators sit higher in the hierarchy. Exponentiation binds tighter than multiplication/division, which bind tighter than addition/subtraction. Parentheses and unary minus are handled at the **primary** level.

$$\begin{aligned} \text{expr} &\rightarrow \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term} \\ \text{term} &\rightarrow \text{term} * \text{factor} \mid \text{term} / \text{factor} \mid \text{factor} \\ \text{factor} &\rightarrow \text{factor}^{\text{primary}} \mid \text{primary} \\ \text{primary} &\rightarrow (\text{expr}) \mid - \text{primary} \mid \text{NUMBER} \end{aligned}$$

3 Implementation

3.1 FLEX Lexer

```
1 %{
2 #include "y.tab.h"
3 #include <stdlib.h>
4 %}
5
6 %%
7
8 [0-9]+(\\.[0-9]+)? {
9     yyval.dval = atof(yytext);
10    return NUMBER;
11 }
12
13 [ \\t] ;
14 \\n return '\\n';
15 "+" return '+';
16 "-" return '-';
17 "*" return '*';
18 "/" return '/';
19 "^" return '^';
20 "(" return '(';
21 ")" return ')';
22 . return yytext[0];
23
24 %%
```

```

25
26 int yywrap() { return 1; }

```

Listing 1: FLEX Lexer

3.2 YACC Parser

```

1 %{
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <math.h>
5
6 void yyerror(const char *s);
7 int yylex();
8 %}
9
10 %union { double dval; }
11
12 %token <dval> NUMBER
13 %type <dval> expr term factor primary
14
15 %%
16
17 input: input line | ;
18
19 line:
20     '\n'
21     | expr '\n' { printf("Result = %g\n", $1); }
22     ;
23
24 expr:
25     expr '+' term { $$ = $1 + $3; }
26     | expr '-' term { $$ = $1 - $3; }
27     | term { $$ = $1; }
28     ;
29
30 term:
31     term '*' factor { $$ = $1 * $3; }
32     | term '/' factor { if ($3 == 0) { yyerror("Division by zero"); $$ = 0; }
33                       else $$ = $1 / $3; }
34     | factor { $$ = $1; }
35     ;
36
37 factor:
38     factor '^' primary { $$ = pow($1, $3); }
39     | primary { $$ = $1; }
40     ;
41
42 primary:
43     '(' expr ')' { $$ = $2; }
44     | '-' primary { $$ = -$2; }
45     | NUMBER { $$ = $1; }
46     ;
47
48 %%
49
50 void yyerror(const char *s) { printf("Syntax Error\n"); }
51
52 int main()
53 {
54     printf("Enter Expression:\n");
55     yyparse();
56     return 0;
57 }

```

Listing 2: YACC Parser

4 Compilation Steps

```
1 yacc -d 210041226_2B_Lab03.y
2 flex 210041226_2B_Lab03.l
3 gcc lex.yy.c y.tab.c -o parser -lm
4 ./parser < input.txt
```

Listing 3: Build and Run

5 Sample Input and Output

5.1 Input (input.txt)

```
1 5+3-2
2 (10-3)+2
3 3*4+2
4 10/2-1
5 2^3
6 2^3^2
7 3.5*2+1.5
8 -5+3
9 (2+3)*(4-1)
```

5.2 Output (output.txt)

```
1 Enter Expression:
2 Result = 6
3 Result = 9
4 Result = 14
5 Result = 4
6 Result = 8
7 Result = 64
8 Result = 8.5
9 Result = -2
10 Result = 15
```

6 Conclusion

The YACC-based parser successfully evaluates arithmetic expressions with correct operator precedence. FLEX handles tokenization and passes tokens to YACC, which applies the grammar rules to compute results. Floating-point support and unary minus were implemented as extensions to the base grammar.